

CSE321 Theory Assignment 01

A process is divided into multiple threads in a multi-threaded system. Here it shares the same memory space, like it has the same heap, global variables, and file descriptors. But each thread contains its own program counter, registers, and call stack. It assigns one thread per connection if the multi-threaded server gets concurrent requests. Threads work independently.

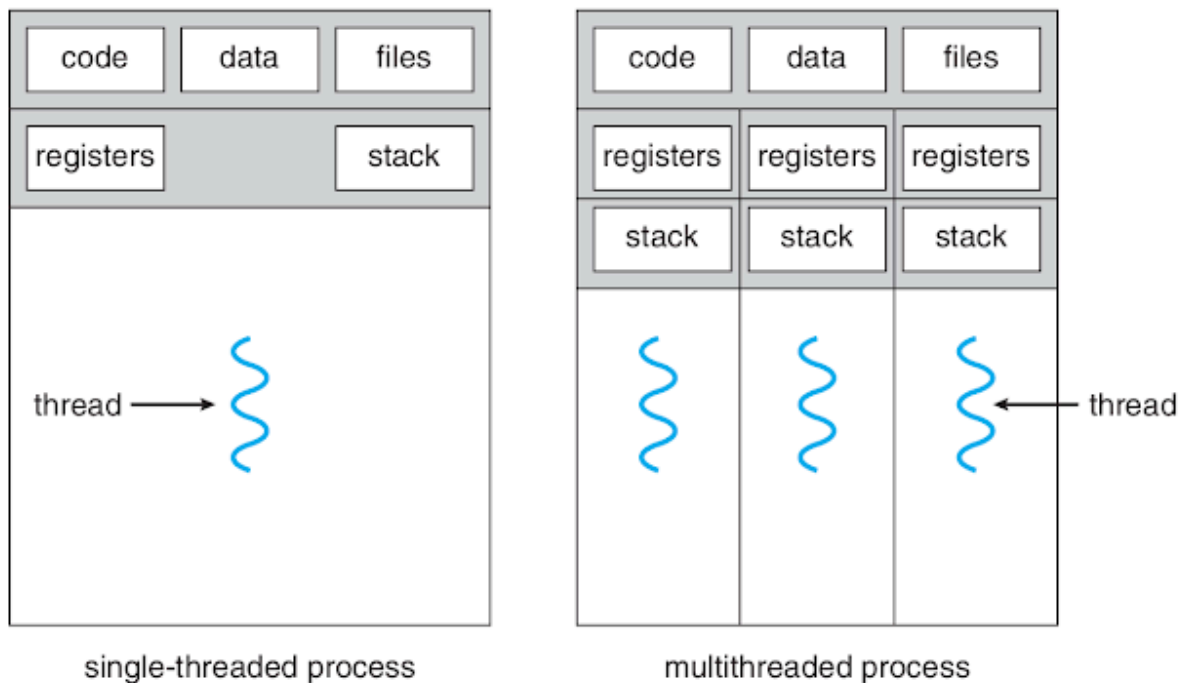


Fig. 1: Operating System: Threads and Concurrency

The OS scheduler works to ensure that all threads make progress, even when some are threads are waiting for I/O. The OS does it through a preemptive scheduler, where threads will have context switching. Some common algorithm that follows preemptive way are :

1. RoundRobin where each thread gets a fixed time quantum
2. MLFQ (Multi-Level Feedback Queue) where each threads follows a fixed amount of quantum time along with I/O-bound

When the scheduler gives the CPU attention from one thread to another thread, we call it context switching. When a thread switches CPU attention, it saves its working status. So when next it gets CPU attention, it can continue its work from there. The program counter saves where it will work next. Also registers, the stack pointer saves the thread's CPU state. Typically a context switch takes 1–10 microseconds. While context switching, the CPU remains idle; it does not do any work.

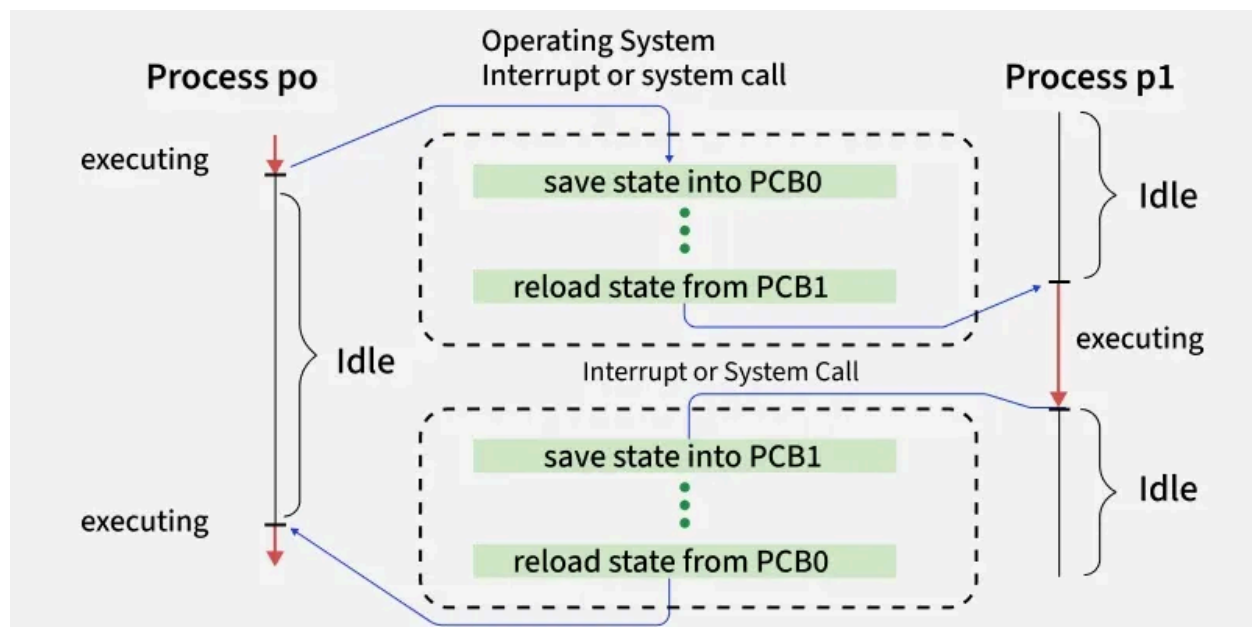


Fig. 2: Context Switching in Operating System

As threads share the same memory and concurrent access is needed, they may get into race conditions. To solve this issue, synchronization has been used. Synchronization helps with ordering and mutual exclusion. For ordering, semaphore is being used, and for mutual exclusion, mutex is being used.

1. Mutex (Mutual Exclusion Lock): Here it locks the critical section. Only one thread can execute the critical section at a time. Others remain locked.
2. Semaphore: It works like a counting primitive. It is useful for controlling access.

Here true parallelism is maintained via parallel threads. Blocking the thread is logically easy. But here it can cause issues like race conditions, deadlocks, and priority inversion. Also sleeping threads waste stack memory.

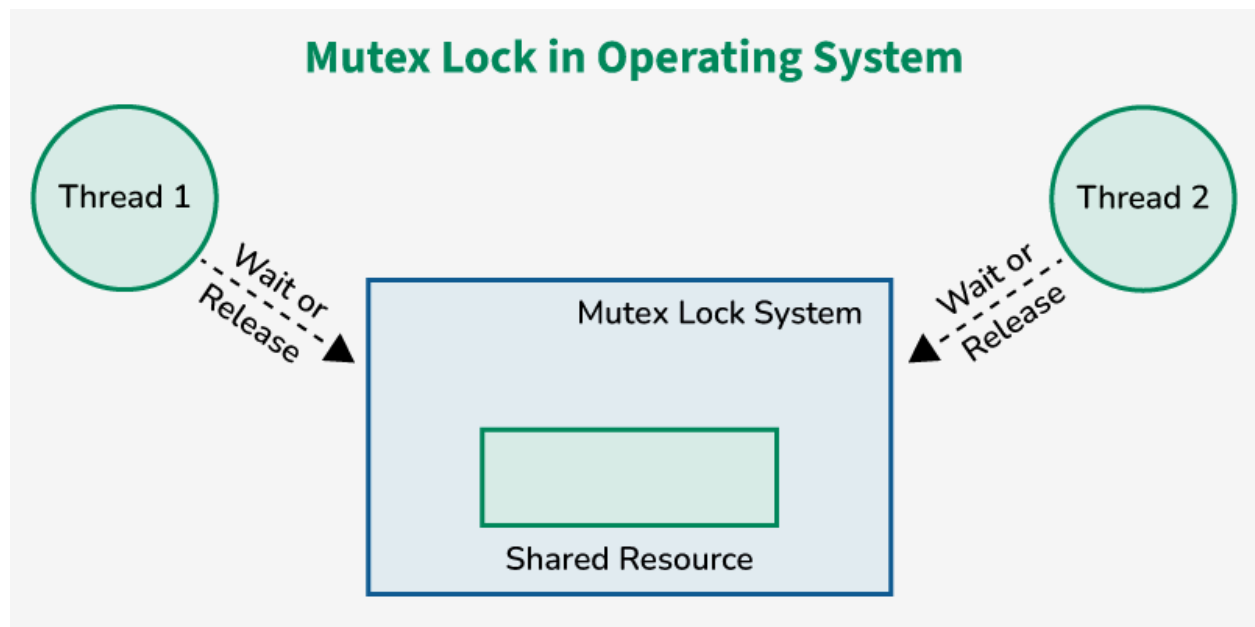


Fig. 3: Mutex

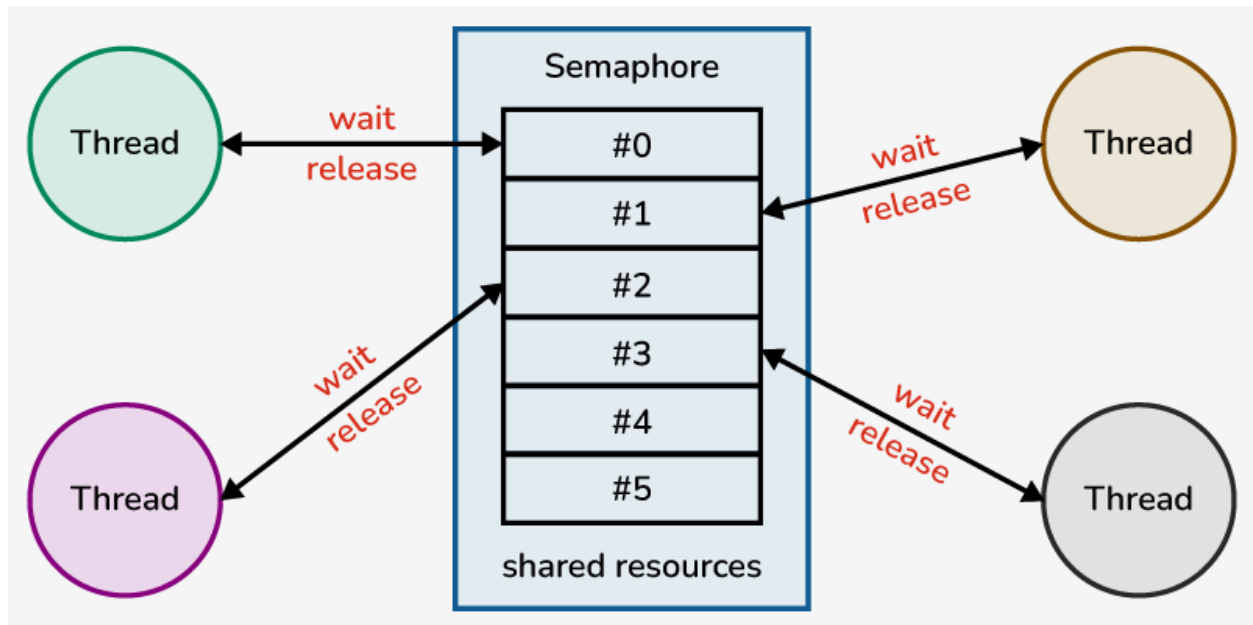


Fig. 4: Semaphore

Event-driven systems use a single thread running an infinite loop. It continuously polls for completed events and dispatches their callbacks. Node.js runs JavaScript on a single main thread powered by V8 (JIT engine). It's backed by libuv, which handles async I/O via OS primitives. libuv also maintains an internal thread pool for operations that cannot be made async at the OS level.

The Node.js event loop runs through six phases per tick:

1. Timers: runs expired `setTimeout` and `setInterval` callbacks
2. Pending Callbacks handle deferred I/O error callbacks
3. Idle: wait for new event
4. Poll: fetches new I/O events from the OS
5. Check: runs `setImmediate()`

6. Close Callbacks

After an async task finishes, Callbacks run a function. But too many nested callbacks create callback hell. It has been improved by Promises. It is a cleaner way to handle results using `.then()` and `.catch()`. Async/await is the simplest, as it lets you write async code like normal code.

Event-driven systems like Node.js use very little memory and avoid race conditions since they run in a single thread. However, any heavy CPU task can block the event loop and slow everything down. And debugging async code can be difficult.

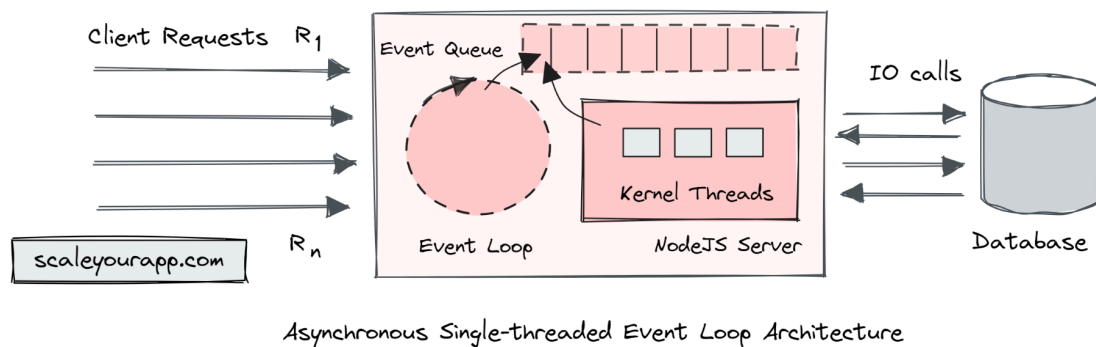


Fig. 5: Single-threaded Event Loop Architecture for Building Asynchronous, Non-Blocking, Highly Concurrent Real-time Services

A thread waits until the operation finishes in blocking I/O. While waiting, it also holds memory and resources. In non-blocking I/O, the thread starts the task and continues executing. It gets notification later when it completes. This allows systems like Node.js to handle thousands of concurrent operations using a single thread.

Event-driven systems like Node.js are very efficient for I/O tasks like database calls, APIs, WebSockets. They can handle many users at once with low memory. Companies like LinkedIn and PayPal improved speed using this approach. Multithreading is better because it uses multiple cores to do work faster For CPU-bound tasks.

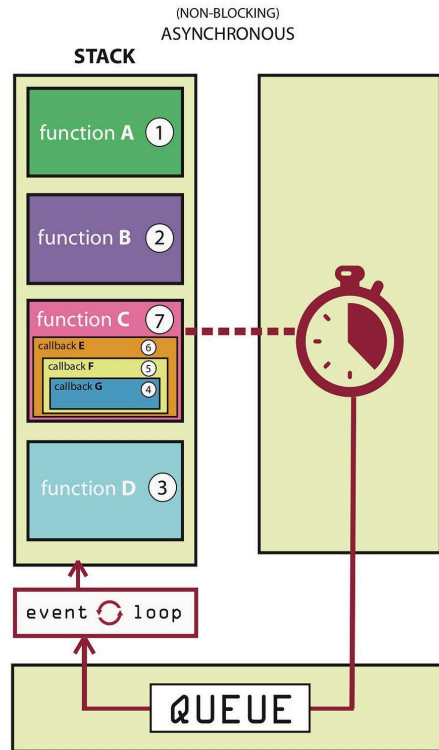


Fig. 6: Asynchronous Programming in JavaScript: Callbacks, Promises, and Async/Await

Each thread needs its own stack. Because of that under high concurrency, thread systems consume large memory. And it also faces context-switching overhead, which reduces throughput. On the other hand, Node.js works efficiently for I/O by using a single-threaded event loop. For this execution, memory depends on only the active requests, not the whole connection. It also maintains lower and more stable tail latency (p99). But threaded systems see latency spikes.

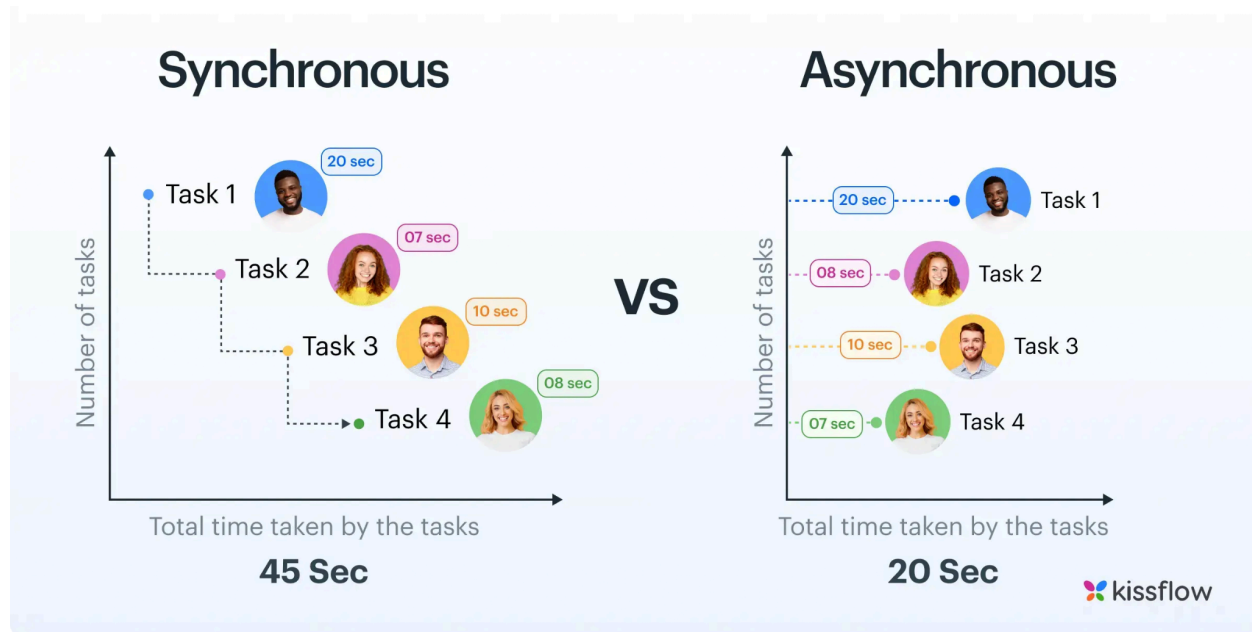


Fig. 7: Asynchronous Programming and Synchronous Programming

For REST API servers and API gateways Event-driven architectures are best. That calls to databases and downstream services like real-time applications such as live chat and collaborative editors. And also in multiplayer games that maintain large numbers of persistent WebSocket connections and microservices

Multi-threaded architectures are more suitable for computational systems. For example: video and image processing, cryptographic systems, and machine learning works. In practice, modern production systems often adopt a hybrid strategy.

For CPU-bound tasks, multithreading works, as it is using multiple cores for true parallelism. But it may add issues like synchronization and race conditions. Event-driven is a non-blocking model with low overhead that uses a single thread. Modern systems

mostly combine both event-driven layers and multithreaded services for handling high concurrency and heavy computation.

References

1. <https://medium.com/@merveyaydin9/processes-and-threads-the-pillars-of-modern-computing-301290277b3d>
2. <https://www.geeksforgeeks.org/operating-systems/multithreading-in-operating-system/>
3. <https://towardsdev.com/thread-synchronization-in-java-73116ee58ac9>
4. <https://www.geeksforgeeks.org/operating-systems/race-condition-in-operating-systems/>
5. Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems* (4th ed.). Pearson. Chapters 2–3.
6. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley. Chapters 4–7.